МЕЖДУНАРОДНЫЙ
ВОСТОЧНО-ЕВРОПЕЙСКИЙ
КОЛЛЕДЖ

ДИНАМИЧЕСКИЕ ДАННЫЕ • ОБЪЕКТЫ • МАССИВЫ

Проектирование и разработка интерфейсов пользователя

Преподаватель

Коляда Никита Владимирович

Тема 5

ПЛАН ЗАНЯТИЯ



1. Объекты и массивы

Структуры данных JS, свойства объектов и массивы каталога.



2. Шаблонные строки

Синтаксис обратных кавычек ` ` , интерполяция ${} и функции разметки.



3. Архитектура рендера

Оптимизация DOM, map().join(''), разделение data.js и main.js.



4. Практика: витрина книг

Генерация каталога, условные бейджи и обработка пустого списка.



5. Итоги занятия и разбор типичных ошибок

Разбор типичных ошибок верстки, методы отладки и закрепление материала

js/main.js (объект сущности)

```
// Объект — это коллекция пар 'ключ: значение'
const book = {
  id: 101,
  title: 'Чистый код',
  author: 'Роберт Мартин',
  price: 1250,
  inStock: true
};

// Доступ через точку (приоритетный способ):
console.log(book.title);  // 'Чистый код'
console.log(book.price);  // 1250

// Доступ через скобки (для динамических ключей):
const field = 'author';
console.log(book[field]); // 'Роберт Мартин'
```

АНАТОМИЯ JS-ОБЪЕКТА

- **Синтаксис литерала объекта { }**
  Компактная запись, группирующая связанные параметры сущности каталога.
- **Ключи и разнородные значения**
  В одном объекте хранятся числа (price), строки (title) и булевы флаги (inStock).
- **Точечная нотация obj.property**
  Самый быстрый и наглядный способ обращения к полям объекта.
- **Скобочная нотация obj['key']**
  Позволяет считывать поля, имя которых хранится в переменной.

Как устроен объект в JavaScript и как считывать его свойства?

js/main.js (методы массивов)

```
// Массив — упорядоченный список значений
const genres = ['Фронтенд', 'Архитектура', 'Алгоритмы'];

// 1. Нумерация элементов с 0:
console.log(genres[0]); // 'Фронтенд'
console.log(genres.length);  // 3 (количество)

// 2. Добавление в конец через push():
genres.push('Базы данных');
console.log(genres.length);  // 4

// 3. Проверка наличия через includes():
console.log(genres.includes('Алгоритмы')); // true
```

СПИСКИ И МЕТОДЫ РАБОТЫ

- **Индексация с нуля [0]**
  Первый элемент всегда имеет индекс 0, последний — array.length - 1.
- **Свойство .length**
  Автоматически отражает текущее точное число элементов в массиве.
- **Метод push()**
  Добавляет новый элемент в самый конец массива, увеличивая его размер.
- **Массив как основа витрины**
  Все товары интернет-магазина или статьи блога хранятся в виде единого массива.

Как массивы организуют упорядоченные коллекции элементов?

js/data.js (модель каталога)

```
const booksCatalog = [
  {
    id: 1, title: 'Чистый JavaScript',
    author: 'И. Смирнов', price: 1100,
    img: 'img/book1.jpg', sale: true
  },
  {
    id: 2, title: 'Паттерны UI',
    author: 'Е. Васильева', price: 1450,
    img: 'img/book2.jpg', sale: false
  }
];
```

РЕАЛЬНЫЕ ДАННЫЕ ВЕБ-ПРИЛОЖЕНИЯ

- **Каждый объект — одна карточка**
  Один элемент массива содержит все данные конкретной книги каталога.
- **Уникальный ключ id**
  Обязательное числовое поле для поиска, открытия деталей и добавления в корзину.
- **Унифицированная схема полей**
  Все объекты обладают одинаковым набором ключей (title, price, img, sale).
- **Масштабируемость**
  Чтобы добавить 100 новых книг, нужно лишь добавить 100 объектов в массив без правки HTML.

Какая структура данных используется для хранения каталога товаров?

js/main.js (способы перебора)

```
// 1. Классический цикл for (по индексам)
for (let i = 0; i < books.length; i++) {
  console.log(books[i].title);
}

// 2. Современный цикл for...of (прямой доступ к объекту)
for (const item of books) {
  console.log(item.title, item.price);
}

// 3. Функциональный forEach (декларативный стиль)
books.forEach(item => {
  console.log(`Книга: ${item.title}`);
});
```

ВЫБОР ОПТИМАЛЬНОГО ЦИКЛА

- **Цикл for...of для генерации**
  Идеален для обхода коллекций: нет возни со счетчиками i и выходом за границы.
- **Метод forEach для компактности**
  Принимает стрелочную функцию, делая код коротким и понятным.
- **Классический for при необходимости шага**
  Нужен только когда требуется перепрыгивать через элементы (i += 2).
- **Чистота кода**
  Современные проекты отдают предпочтение for...of и методам высшего порядка.

Какой цикл наиболее удобен для перебора массива объектов?

СХЕМА: ПОТОК ДАННЫХ В DOM

1. ХРАНИЛИЩЕ (DATA)
Массив объектов в data.js

```
const books = [{ id, title, price }]
```

Единый источник правды:

Данные изолированы в файле data.js. Никакой разметки — только чистые свойства сущностей каталога (книги, товары, блюда).

2. ШАБЛОНИЗАЦИЯ
Шаблонная строка в main.js

```
renderCard(b) => `<div class="card">...`
```

Генерация разметки на лету:

Функция-рендерер принимает объект данных и с помощью обратных кавычек подставляет ${book.title} в готовый каркас HTML.

3. МОНТИРОВАНИЕ (DOM)
Вставка в контейнер страницы

```
catalog.innerHTML = htmlString
```

Отображение пользователю:

Браузер считывает готовую HTML-строку и за 1 операцию перерисовывает витрину каталога со всеми карточками.

Как сырые данные из массива трансформируются в видимые карточки на экране?

js/main.js (Template Literals)

```
const title = 'Паттерны проектирования';
const price = 1350;

//  Устаревшая конкатенация строк плюсами:
const oldHtml = '<div class="card"><h3>' + title + '</h3><p>' +
price + ' руб</p></div>';

//  Современные шаблонные строки (обратные кавычки ``):
const newHtml = `
  <article class="book-card">
    <h3>${title}</h3>
    <span class="price">${price} ₽</span>
  </article>
`;
```

ПРЕИМУЩЕСТВА ШАБЛОННЫХ СТРОК

- **Синтаксис обратных кавычек ` `**
  Позволяет писать многострочный HTML-код с сохранением отступов и переносов.
- **Интерполяция выражений ${}**
  Автоматически вычисляет и подставляет любые переменные и тернарные операторы.
- **Исключение ошибок экранирования**
  Внутри кавычек ` ` можно свободно использовать двойные и одинарные кавычки.
- **Читаемость как в HTML**
  Разметка внутри JS-функции выглядит в точности как обычный исходный код HTML5.

Почему интерполяция в обратных кавычках заменила конкатенацию плюсами?

ФУНКЦИЯ RENDERCARD

js/main.js (генератор разметки)

```
// Функция принимает 1 объект и возвращает 1 строку HTML
function renderBookCard(book) {
  return `
    <article class="book-card" data-id="${book.id}">
      <img src="${book.img}" alt="${book.title}">
      <div class="card-info">
        <h3>${book.title}</h3>
        <p class="author">${book.author}</p>
        <span class="price">${book.price} ₽</span>
      </div>
    </article>
  `;
}
```

ШАБЛОНИЗАЦИЯ КОМПОНЕНТА

- **Чистая функция рендеринга**
  На вход получает объект данных — на выходе возвращает готовую строку разметки.
- **Атрибут data-id**
  Связывает визуальный блок в DOM с идентификатором объекта в массиве данных.
- **Переиспользуемость**
  Эту же функцию можно вызывать в каталоге, корзине, поиске и избранном.
- **Легкость редизайна**
  Чтобы поменять вид всех 100 карточек, правится всего 1 функция renderBookCard.

Как изолировать верстку карточки в отдельную функцию-генератор?

js/main.js (производительность DOM)

```
const container = document.querySelector('.catalog');

//  АНТИПАТТЕРН: innerHTML += на каждой итерации (медленно!)
// При каждой итерации браузер заново уничтожает и строит весь DOM!
for (const b of books) {
  container.innerHTML += renderBookCard(b); // Reflow на каждый шаг!
}

//  ПРАВИЛЬНО: склеить строки и вставить ВСЁ ЗА 1 РАЗ
let htmlBuffer = '';
for (const b of books) {
  htmlBuffer += renderBookCard(b);
}
container.innerHTML = htmlBuffer; // 1 перерисовка страницы!
```

ОПТИМИЗАЦИЯ ПЕРЕРИСОВКИ DOM

- **Тяжелая операция Reflow / Repaint**
  Каждая запись в innerHTML заставляет браузер заново пересчитывать геометрию элементов.
- **Сборка буфера в памяти**
  Склейка HTML-строки в переменной происходит мгновенно в оперативной памяти.
- **Единая запись в DOM**
  Запись собранной строки за один прием ускоряет отображение каталога в сотни раз.
- **Альтернатива: map().join('')**
  Декларативный способ склейки массива карточек в единую HTML-строку.

Почему нельзя использовать container.innerHTML += внутри цикла?

js/main.js (сборка каталога)

```
function renderCatalog(itemsList) {
  const catalogContainer = document.querySelector('#catalog-grid');

  // 1. Проверка на пустой результат
  if (!itemsList || itemsList.length === 0) {
    catalogContainer.innerHTML = '<p class="empty">Книги не
найдены</p>';
    return;
  }

  // 2. Генерация разметки для всех элементов
  const cardsHtml = itemsList.map(book =>
renderBookCard(book)).join('');

  // 3. Монтирование в контейнер
  catalogContainer.innerHTML = cardsHtml;
}
```

АРХИТЕКТУРА РЕНДЕРИНГА

- **Поиск целевого контейнера**
  Находит узел #catalog-grid, куда будут вставлены готовые карточки.
- **Защита от пустого списка**
  Если массив пуст, выводится аккуратная информационная заглушка.
- **Элегантный map().join('')**
  Преобразует массив объектов в массив строк HTML и склеивает их в единый текст.
- **Универсальность аргумента**
  Функция отрисовывает как полный каталог, так и отфильтрованные результаты поиска.

Как устроена главная функция отображения каталога renderCatalog()?

js/main.js (условия в шаблоне)

```
function renderBookCard(book) {
  return `
    <article class="book-card">
      <div class="thumb-wrapper">
        <img src="${book.img}" alt="${book.title}">
        <!-- Тернарный оператор: условие ? истина : ложь -->
        ${book.sale ? '<span class="badge sale">Скидка</span>' : ''}
        ${book.inStock ? '' : '<span class="badge out">Под
заказ</span>'}
      </div>
      <h3>${book.title}</h3>
      <span class="price">${book.price} ₽</span>
    </article>
  `;
}
```

ДИНАМИЧЕСКИЕ ЭЛЕМЕНТЫ КАРТОЧКИ

- **Тернарный оператор condition ? A : B**
  Идеально встраивается внутрь ${} прямо в середине шаблонной строки разметки.
- **Условные бейджи скидки**
  Если book.sale === true — генерируется тег плашки, иначе подставляется пустая строка.
- **Условные CSS-классы**
  Позволяет динамически менять классы карточки: class='card ${book.active ? 'active' : ''}'.
- **Чистота без громоздких if**
  Компактная запись сохраняет целостность шаблона карточки без разрыва на части.

Как с помощью тернарного оператора выводить бейджи со скидкой?

МОДУЛЬНОСТЬ КОДА

index.html (подключение модулей)

```
<!-- Порядок подключения скриптов в <head>: -->

<!-- 1. Сначала загружаем данные каталога: -->
<script src="js/data.js" defer></script>

<!-- 2. Затем логику, которая с этими данными работает: -->
<script src="js/main.js" defer></script>

// В js/data.js:
const BOOKS = [ /* массив из 6+ книг */ ];

// В js/main.js:
document.addEventListener('DOMContentLoaded', () => {
  renderCatalog(BOOKS); // Массив BOOKS уже доступен в памяти!
});
```

РАЗДЕЛЕНИЕ ОТВЕТСТВЕННОСТИ

- **Файл js/data.js — Модель**
  Содержит только массив сущностей. Никаких querySelector и слушателей событий.
- **Файл js/main.js — Представление**
  Отвечает за генерацию DOM, анимации, переключение тем и клики мыши.
- **Строгий порядок defer**
  Браузер выполняет defer-скрипты строго по порядку: data.js создаст переменные до main.js.
- **Удобство командной работы**
  Контент-менеджер правит data.js, а фронтенд-разработчик верстает main.js без конфликтов.

Зачем разделять данные и логику приложения на разные файлы?

ДЕСТРУКТУРИЗАЦИЯ

js/main.js (деструктуризация)

```
const book = {
  title: 'Грокаем алгоритмы',
  price: 950,
  author: 'А. Бхаргава'
};

// Извлечение полей в отдельные локальные переменные за 1 шаг:
const { title, price, author } = book;
console.log(title);  // 'Грокаем алгоритмы'
console.log(price);  // 950

// Деструктуризация прямо в аргументах функции рендеринга:
function renderShortCard({ title, price }) {
  return `<div class="mini"><h4>${title}</h4><span>${price}
₽</span></div>`;
}
```

ИЗВЛЕЧЕНИЕ СВОЙСТВ БЕЗ ПОВТОРОВ

- **Синтаксис const { a, b } = obj**
  Создает локальные переменные с именами совпавших ключей объекта.
- **Отказ от постоянных obj.property**
  Избавляет от утомительного многократного повторения book.title, book.author.
- **Деструктуризация в параметрах**
  Функция принимает только нужные свойства ({ title, price }), игнорируя лишние.
- **Значения по умолчанию**
  Можно задать дефолт: const { rating = 5 } = book, если свойство отсутствует.

Как деструктуризация делает код работы со свойствами короче и чище?

Анализ типичных ошибок

```
/*  АНТИПАТТЕРН 1: Хардкод 20 карточек в index.html */
<!-- Чтобы поменять верстку кнопки, придется вручную править 20 мест! -->
<div class="card"><h3>Книга 1</h3><button>Купить</button></div>
<div class="card"><h3>Книга 2</h3><button>Купить</button></div>

/*  АНТИПАТТЕРН 2: Карточки без уникального идентификатора id */
const badData = [{ title: 'Товар' }, { title: 'Товар' }]; // Как их отличить?

/*  ПРАВИЛЬНЫЙ ПОДХОД: Чистые данные + 1 функция-шаблон */
const BOOKS = [{ id: 1, title: 'Книга 1' }, { id: 2, title: 'Книга 2' }];
renderCatalog(BOOKS); // Генерирует всё автоматически!
```

АРХИТЕКТУРНЫЕ ТРЕБОВАНИЯ

- **Отказ от ручного копипаста карточек**
  Разметка каталога обязана строиться динамически из массива данных.
- **Обязательный уникальный id**
  Каждая запись каталога должна иметь числовой или строковый идентификатор.
- **Иммутабельность исходных данных**
  Нельзя случайно перезаписывать или удалять поля в глобальном массиве BOOKS.
- **Проверка обязательных полей**
  Шаблонизатор обязан корректно обрабатывать карточки, у которых нет картинки или цены.

Почему хардкод HTML-карточек в коде страницы недопустим в продакшене?

СХЕМА: МОДУЛЬНАЯ СТРУКТУРА

1. МОДУЛЬ ДАННЫХ
js/data.js (Только массив)

```
window.booksData = [...]
```

Хранит чистую структуру данных. Легко редактировать цены и названия без риска сломать логику рендеринга.

2. МОДУЛЬ ЛОГИКИ
js/main.js (Рендерер и события)

```
renderCatalog(booksData)
```

Принимает массив данных, строит разметку и подписывается на события фильтрации и кликов.

3. ТОЧКА ВХОДА
index.html (Монтирование)

```
<script src='data.js' defer>
<script src='main.js' defer>
```

Подключает модули в строгом порядке: сначала данные, затем логика, которая с ними работает.

Как взаимодействуют файлы data.js, main.js и index.html в проекте?

ПРАКТИКА: ШАГ 1

js/data.js (база книг)

```
// Массив каталога книг (6 реалистичных записей)
const BOOKS_DATA = [
  { id: 1, title: 'Чистый код', author: 'Р. Мартин', price: 1250,
img: 'img/book1.jpg', sale: true },
  { id: 2, title: 'Совершенный код', author: 'С. Макконнелл', price:
1800, img: 'img/book2.jpg', sale: false },
  { id: 3, title: 'JavaScript: Карманный справочник', author: 'Д.
Флэнаган', price: 850, img: 'img/book3.jpg', sale: true },
  { id: 4, title: 'Паттерны разработки', author: 'Э. Гамма', price:
1500, img: 'img/book4.jpg', sale: false }
];
```

ПОДГОТОВКА МОСК-ДАННЫХ

- **Реалистичные названия и цены**
  Тестовые данные должны соответствовать настоящему интернет-магазину книг.
- **Строгая структура полей**
  Каждая запись содержит id, title, author, price, img и логический флаг sale.
- **Готовность к расширению**
  Легко добавить 5-ю и 6-ю книгу: структура каталога подхватит их автоматически.
- **Глобальная видимость**
  Объявленная переменная BOOKS_ DATA становится доступна в скрипте main.js.

Как наполнить файл data.js качественными тестовыми данными (Mock Data)?

js/main.js (шаблонизатор)

```
function createCardHtml(book) {
  return `
    <article class="book-card" data-id="${book.id}">
      <div class="book-thumb">
        <img src="${book.img}" alt="${book.title}">
        ${book.sale ? '<span class="badge-sale">Акция</span>' : ''}
      </div>
      <div class="book-content">
        <h3 class="book-title">${book.title}</h3>
        <p class="book-author">${book.author}</p>
        <div class="book-footer">
          <span class="book-price">${book.price} ₽</span>
          <button class="btn-buy">В корзину</button>
        </div>
      </div>
    </article>
  `;
}
```

РЕАЛИЗАЦИЯ ШАБЛОНА

- **Семантический тег article**
  Оформляет карточку книги как самостоятельную смысловую публикацию.
- **Интерполяция автора и цены**
  Строка ${book.author} и ${book.price} ₽ заполняются автоматически.
- **Бейдж акции по условию**
  Тернарный оператор добавляет плашку 'Акция' только на книги со скидкой.
- **Кнопка покупки**
  Кнопка 'В корзину' готова к будущей обработке кликов пользователей.

Как написать функцию шаблона карточки с подстановкой данных?

js/main.js (запуск рендеринга)

```
function renderBooks(booksList) {
  const grid = document.querySelector('#books-grid');

  // Собираем HTML всех карточек в одну строку:
  const allCardsHtml = booksList
    .map(b => createCardHtml(b))
    .join('');

  // Вставляем все карточки за 1 обращение к DOM!
  grid.innerHTML = allCardsHtml;
}

// Запуск при готовности страницы:
document.addEventListener('DOMContentLoaded', () => {
  renderBooks(BOOKS_DATA); // Витрина оживает!
});
```

ОТРИСОВКА ВСЕЙ ВИТРИНЫ

- **Поиск целевого контейнера #books-grid**
  Находит CSS Grid-сетку, сверстанную на предыдущем вебинаре.
- **Метод map() для трансформации**
  Превращает массив из 6 объектов в массив из 6 строк HTML-разметки.
- **Метод join('') для склейки**
  Объединяет все фрагменты в единый монолитный текст без разделителей.
- **Старт в DOMContentLoaded**
  Отображает полный каталог моментально при открытии веб-страницы.

Как соединить массив данных и функцию карточки для отрисовки витрины?

ПРАКТИКА: ШАГ 4

js/main.js (Empty State)

```
function renderBooks(booksList) {
  const grid = document.querySelector('#books-grid');

  // Проверка: пуст ли полученный массив?
  if (!booksList || booksList.length === 0) {
    grid.innerHTML = `
      <div class="empty-state">
        <p class="empty-title">Книги не найдены</p>
        <p class="empty-desc">Попробуйте изменить поисковый
запрос</p>
      </div>
    `;
    return; // Выход из функции
  }

  grid.innerHTML = booksList.map(b => createCardHtml(b)).join('');
}
```

ОБРАБОТКА КРАЕВЫХ СИТУАЦИЙ

- **Паттерн Empty State (Пустое состояние)**
  Пользователь никогда не должен видеть пустой белый экран при отсутствии товаров.
- **Ранний выход return**
  Прерывает выполнение функции, не запуская лишний цикл map().
- **Понятная подсказка для человека**
  Информирует пользователя, что товаров нет, и советует сбросить фильтры.
- **Профессиональный стандарт UX**
  Обязательное требование ко всем экзаменационным проектам.

Как обработать ситуацию, когда по запросу ничего не найдено?

ОБОБЩЕНИЕ

Формула динамического UI

```
/* Формула любого современного динамического каталога: */
/* 1. ИСТОЧНИК ДАННЫХ (Data Source): */
   const items = [{ id, title, price, ... }];

/* 2. ШАБЛОНИЗАТОР (View Component): */
   const renderCard = (item) => `<article>...</article>`;

/* 3. ОПЕРАЦИЯ МОНТИРОВАНИЯ (Mounting to DOM): */
   container.innerHTML = items.map(renderCard).join('');

/* 4. РЕАКЦИЯ НА ИЗМЕНЕНИЯ: */
   Изменился массив данных -> Вызов render -> Обновление экрана
```

ЕДИНЫЙ ИСТОЧНИК ПРАВДЫ

- **Данные первичны, разметка вторична**
  Интерфейс строится строго на основе данных, хранящихся в объектах.
- **Декларативное описание витрины**
  Разработчик описывает вид 1 карточки, а JavaScript масштабирует ее на весь список.
- **Мгновенное обновление**
  Добавление или удаление объекта из массива синхронно обновляет интерфейс.
- **Фундамент для фреймворков**
  Этот же принцип лежит в основе React, Vue и современных SPA-приложений.

В чем заключается фундаментальная формула динамического рендеринга?

РАЗБОР ОШИБОК

4 ТИПИЧНЫЕ ОШИБКИ ПРИ РАБОТЕ С ДАННЫМИ И РЕНДЕРИНГОМ

Забыли подключить data.js перед main.js

✕ **Ошибка:** main.js пытается прочитать массив BOOKS, но он еще не объявлен. Ошибка ReferenceError.

✓ **Решение:** В секции `<head>` подключать `<script src='data.js'>` СТРОГО перед main.js.

Склеивание innerHTML += в теле цикла

✕ **Ошибка:** Приводит к многократным тяжелым перерисовкам страницы и лагам при 50+ товарах.

✓ **Решение:** Склеивать строку в переменной-буфере или использовать массив .map().join('').

Опечатка в имени ключа (${book.Prise})

✕ **Ошибка:** Ключи чувствительны к регистру. Несуществующее свойство выводит в карточке 'undefined ₽'.

✓ **Решение:** Строго сверять имена свойств с исходным объектом: book.price.

Забытые обратные кавычки (" вместо ` `)

✕ **Ошибка:** Использование одинарных кавычек выводит на экран буквальный текст '${book.title}'.

✓ **Решение:** Для интерполяции переменных ВСЕГДА использовать обратные кавычки (` `).

На каких ошибках чаще всего спотыкаются при динамической генерации карточек?

МЕЖДУНАРОДНЫЙ
ВОСТОЧНО-ЕВРОПЕЙСКИЙ
КОЛЛЕДЖ

ДО ВСТРЕЧИ!

УВИДИМСЯ НА СЛЕДУЮЩЕМ ЗАНЯТИИ